

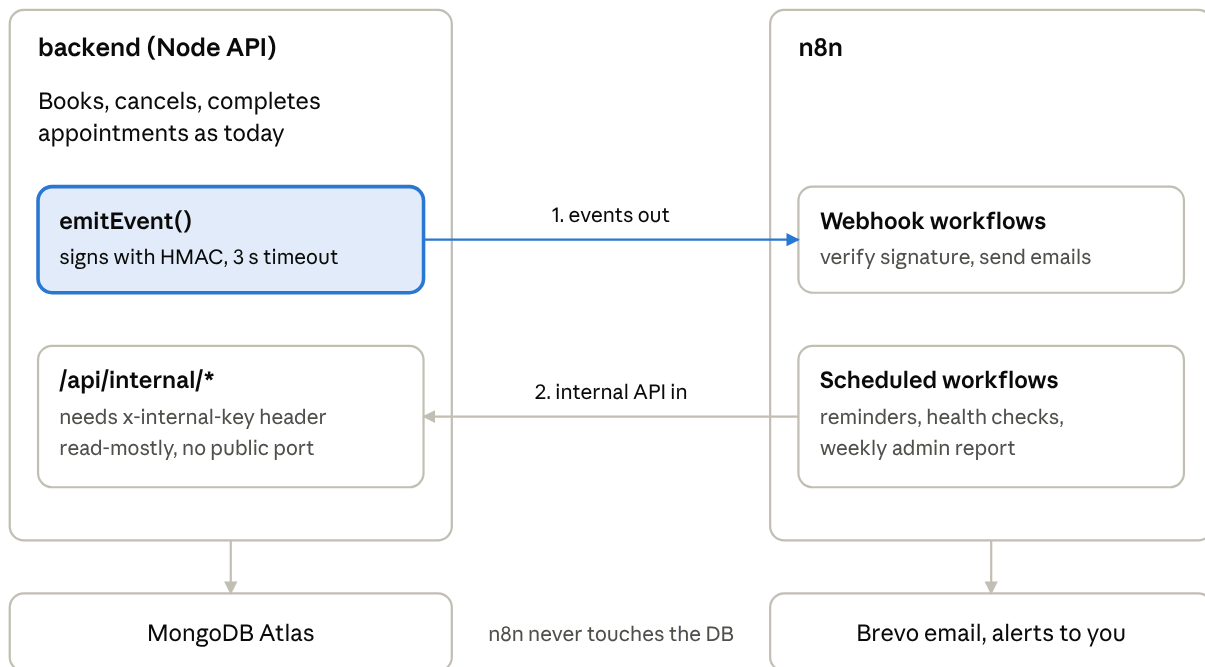
DocOp Phase 2: Production-Grade n8n Automation

Sep 30, 2026 · @Dilina

The design

What makes this "production grade" is not the number of workflows. It is **how n8n connects to your app**: two narrow, authenticated doors, and a backend that keeps working even when n8n is down.

Backend and n8n talk through two secured doors



DocOp + n8n · 2 connections, both authenticated

- **Door 1, events out:** when something happens (booked, paid, cancelled), the backend sends a small signed message to an n8n webhook.
 - Why: the backend does not wait for emails. Booking stays fast, and if n8n is down, booking still works.
- **Door 2, internal API in:** scheduled workflows ask the backend for data (such as tomorrow's appointments) through `/api/internal/*`, protected by a secret key.

- Why: n8n never gets the database password. The backend stays the only thing that reads or writes MongoDB, so all your validation and rules still apply.
- **Secrets are checked both ways:** the backend signs events with HMAC; the backend checks `x-internal-key` on internal routes.
 - Why: anyone who finds the webhook URL cannot fake a "payment done" event.

This is the part to explain in interviews: event-driven, decoupled, secured, failure-tolerant.

Step 1: Harden n8n in docker-compose.yml

First, make n8n itself behave like a production service: fixed version, bounded disk use, shared secrets.

1. **Find your current n8n version** and write it down:

```
docker compose exec n8n n8n --version
```

2. **Add two shared secrets** to the root `.env` (and to the root `.env.example` with placeholder values):

```
# backend -> n8n: signs event webhooks
DOCOP_WEBHOOK_SECRET=paste-output-of-openssl-rand-hex-32
# n8n -> backend: key for /api/internal/*
DOCOP_INTERNAL_KEY=paste-another-openssl-rand-hex-32
```

- Why two different secrets: if one leaks, only one direction is exposed, and you can change it alone.
3. **Replace the n8n service** with this (put your version number in place of `X.Y.Z`):

```
n8n:
  image: n8nio/n8n:X.Y.Z          # pinned, never :latest
  environment:
    DB_TYPE: postgresdb
    DB_POSTGRESDB_HOST: postgres
    DB_POSTGRESDB_DATABASE: n8n
    DB_POSTGRESDB_USER: n8n
    DB_POSTGRESDB_PASSWORD: ${N8N_DB_PASSWORD}
    N8N_ENCRYPTION_KEY: ${N8N_ENCRYPTION_KEY}
    GENERIC_TIMEZONE: Asia/Colombo
    TZ: Asia/Colombo
    # keep only 7 days of execution history
    EXECUTIONS_DATA_PRUNE: "true"
```

```

EXECUTIONS_DATA_MAX_AGE: "168"
# privacy + fewer surprises
N8N_DIAGNOSTICS_ENABLED: "false"
N8N_PERSONALIZATION_ENABLED: "false"
# let workflows read the two secrets via $env
N8N_BLOCK_ENV_ACCESS_IN_NODE: "false"
DOCOP_WEBHOOK_SECRET: ${DOCOP_WEBHOOK_SECRET}
DOCOP_INTERNAL_KEY: ${DOCOP_INTERNAL_KEY}
DOCOP_API_URL: http://backend:8000
ports:
  - "5678:5678"
volumes:
  - n8n-data:/home/node/.n8n
depends_on:
  postgres:
    condition: service_healthy
restart: unless-stopped
healthcheck:
  test: ["CMD-SHELL", "wget -qO- http://localhost:5678/healthz || exit
1"]
  interval: 30s
  retries: 3

```

4. Give the backend the same secrets. Under the `backend` service's `environment:` add:

```

DOCOP_WEBHOOK_SECRET: ${DOCOP_WEBHOOK_SECRET}
DOCOP_INTERNAL_KEY: ${DOCOP_INTERNAL_KEY}
N8N_WEBHOOK_BASE: http://n8n:5678/webhook

```

5. Restart: `docker compose up -d`.

Why each setting:

- **Pinned image:** `:latest` can jump a major version on any restart and break your workflows without warning. A pinned version changes only when you decide.
- **EXECUTIONS_DATA_PRUNE + MAX_AGE:** n8n saves every run. Without pruning, Postgres grows forever and eventually fills the disk.
- **TZ and GENERIC_TIMEZONE:** a "9 am reminder" fires at 9 am Sri Lanka time, not UTC.
- **Secrets as env vars, not typed into nodes:** they live in `.env` (never in Git) and in exported workflow JSON they appear only as `$env.DOCOP_INTERNAL_KEY`, so you can commit workflows safely.

- **DOCOP_API_URL**: one place to change the backend address when you deploy; workflows use `{{env.DOCOP_API_URL}}`.
- **Healthcheck + restart**: same as the backend, Docker restarts n8n if it hangs.
- **Before deploying publicly** (not needed on your laptop): put n8n behind HTTPS and set `N8N_HOST`, `N8N_PROTOCOL=https` and `WEBHOOK_URL`.

Step 2: Backend side (4 new files, a few one-line hooks)

The backend gets one function to send events and one protected route group for n8n. No other logic moves.

2a. `backend/services/eventService.js`: send signed events

```
import crypto from "crypto";

const BASE = process.env.N8N_WEBHOOK_BASE;           //
http://n8n:5678/webhook
const SECRET = process.env.DOCOP_WEBHOOK_SECRET;

// Fire-and-forget: never throws, never slows the user's request.
export function emitEvent(type, data) {
  if (!BASE || !SECRET) return;                      // n8n not configured:
  skip quietly
  const body = JSON.stringify({ id: crypto.randomUUID(), type, occurredAt:
    new Date().toISOString(), data });
  const ts = Date.now().toString();
  const signature = crypto.createHmac("sha256",
    SECRET).update(`${ts}.${body}`).digest("hex");

  fetch(`${BASE}/docop-events`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "x-docop-timestamp": ts,
"x-docop-signature": signature },
    body,
    signal: AbortSignal.timeout(3000),
  }).catch((err) => console.error(`[event] ${type} not delivered:`,
    err.message));
}
```

- **One webhook URL, many event types** (`docop-events`). Why: n8n routes by `type` with a Switch node, so a new event never needs a new URL.
- **id (UUID) on every event**. Why: if an event is ever delivered twice, n8n can spot the repeat.
- **HMAC over `timestamp.body`**. Why: the signature proves the backend sent it; the timestamp stops someone replaying an old captured message.
- **No `await`, 3 s timeout, `.catch`**. Why: if n8n is down or slow, booking still succeeds instantly. Automation is a side effect, never a blocker.

2b. `backend/middleware/authInternal.js`: protect n8n's door

```
import crypto from "crypto";

export default function authInternal(req, res, next) {
  const given = Buffer.from(req.headers["x-internal-key"] || "");
  const expected = Buffer.from(process.env.DOCOP_INTERNAL_KEY || "");
  if (!expected.length || given.length !== expected.length ||
!crypto.timingSafeEqual(given, expected)) {
    return res.status(401).json({ success: false, message: "Unauthorized" });
  }
  next();
}
```

- **`timingSafeEqual` instead of `===`**. Why: `===` stops at the first wrong character, so response times can leak the key one character at a time.
- **Empty key = always refuse**. Why: a missing env var must lock the door, not open it.

2c. `backend/routes/internalRoute.js`: what n8n may ask for

```
import express from "express";
import authInternal from "../middleware/authInternal.js";
import appointmentModel from "../models/appointmentModel.js";
import { isValidSlotDate } from "../services/slotService.js";

const router = express.Router();
router.use(authInternal);

// Appointments on a date that still need a reminder. ?date=1_10_2026
router.get("/appointments/due-reminders", async (req, res) => {
  const { date } = req.query;
```

```

    if (!isValidSlotDate(date)) return res.status(400).json({ success: false,
message: "Bad date" });
    const rows = await appointmentModel
      .find({ slotDate: date, cancelled: false, isCompleted: false,
reminderSentAt: null })
      .lean();
    res.json({ success: true, appointments: rows.map((a) => ({
      id: a._id, slotDate: a.slotDate, slotTime: a.slotTime,
      patientName: a.userData?.name, patientEmail: a.userData?.email,
      doctorName: a.docData?.name,
    }))) });
  });

// Mark a reminder as sent. Only succeeds once per appointment.
router.post("/appointments/:id/reminder-sent", async (req, res) => {
  const updated = await appointmentModel.findOneAndUpdate(
    { _id: req.params.id, reminderSentAt: null },
    { $set: { reminderSentAt: new Date() } }
  );
  res.json({ success: true, firstTime: Boolean(updated) });
});

// Numbers for the weekly admin report.
router.get("/stats/weekly", async (req, res) => {
  const since = Date.now() - 7 * 24 * 60 * 60 * 1000;
  const recent = { date: { $gte: since } };
  const [booked, cancelled, completed, paid] = await Promise.all([
    appointmentModel.countDocuments(recent),
    appointmentModel.countDocuments({ ...recent, cancelled: true }),
    appointmentModel.countDocuments({ ...recent, isCompleted: true }),
    appointmentModel.countDocuments({ ...recent, payment: true }),
  ]);
  res.json({ success: true, booked, cancelled, completed, paid });
});

export default router;

```

- **Returns only the fields a workflow needs** (not the whole `userData` with phone, address and so on). Why: less personal data in n8n's execution logs.
- **`isValidSlotDate` reused**. Why: same input rules as the rest of your API.
- **`reminderSentAt: null` inside the update filter**. Why: two runs racing each other can never both mark it; this is the same atomic trick as your `reserveSlot`.

2d. Wire it up

- `appointmentModel.js`: add `reminderSentAt: { type: Date, default: null }`,
- `server.js`: `import internalRouter from "./routes/internalRoute.js";` and `app.use("/api/internal", internalRouter);`
- Call `emitEvent` right **after** the database write succeeds:

Where	Add
<code>userController.bookAppointment</code> , after <code>newAppointment.save()</code>	<code>emitEvent("appointment.booked", { appointmentId: newAppointment._id, patientName: userData.name, patientEmail: userData.email, doctorName: docData.name, doctorEmail: docData.email, slotDate, slotTime })</code>
<code>userController.verifyMockPayment</code> , after payment is saved	<code>emitEvent("appointment.paid", { ... same fields, amount })</code>
<code>userController.cancelAppointment</code> , <code>doctorController.appointmentCancel</code> , <code>adminController.appointmentCancel</code>	<code>emitEvent("appointment.cancelled", { ..., cancelledBy: "patient" / "doctor" / "admin" })</code>
<code>doctorController.appointmentComplete</code>	<code>emitEvent("appointment.completed", { ... })</code>

- **After a successful write only.** Why: never tell the world about a booking that failed to save.
- **Your payment emails** (`sendEmail` in `verifyMockPayment`) move to n8n in workflow 2 below. Keep the old code until the n8n version works, then delete it.

Test it: `curl http://localhost:8000/api/internal/stats/weekly` should give **401**. The same with `-H "x-internal-key: <your key>"` should give numbers.

Step 3: Build the workflows, in this order

Five workflows, easiest first. Each one works on its own, so you can stop after any of them and still have something to show.

One-time setup in n8n: go to **Credentials** → **Add** → **Brevo** and paste your Brevo API key.
 Why: n8n encrypts it with your `N8N_ENCRYPTION_KEY`, and exported workflows only reference it by name.

Workflow 1: Health monitor (30 min)

1. **Schedule Trigger:** every 5 minutes.
2. **HTTP Request:** `GET {{$env.DOCOP_API_URL}}/api/health`. Under Settings, turn on **Continue on fail** and set the timeout to 5000 ms.
3. **IF:** `{{ $json.status }}` is not equal to `ok`.
4. **Brevo, true branch:** email yourself "DocOp API is down", including `{{ $json.db }}`.
 - Why: you hear about an outage before your users do. This is basic uptime monitoring.
 - Why "Continue on fail": if the backend is completely dead, the request errors; without this, the workflow crashes instead of alerting you.

Workflow 2: Event router + appointment emails (the main one)

1. **Webhook:** method POST, path `docop-events`, Respond: **Using 'Respond to Webhook' node**.
2. **Crypto:** action **Hmac**, type SHA256, secret `{{ $env.DOCOP_WEBHOOK_SECRET }}`, value `{{ $json.headers['x-docop-timestamp'] }}`. `{{ JSON.stringify($json.body) }}`, output property `expected`.
3. **IF "valid?"**, both must be true:
 - `{{ $json.expected }}` equals `{{ $json.headers['x-docop-signature'] }}`
 - `{{ Date.now() - Number($json.headers['x-docop-timestamp']) }}` is smaller than `300000` (5 minutes)
4. **False branch → Respond to Webhook:** status **401**. Stop.
5. **True branch → Respond to Webhook:** status **202**. Then continue.
6. **Switch** on `{{ $json.body.type }}`, one output each for `appointment.booked`, `appointment.paid`, `appointment.cancelled`, `appointment.completed`.
7. On each output, a **Brevo** node (or two: one to the patient, one to the doctor), with the text built from `{{ $json.body.data.patientName }}`, `slotDate`, `slotTime` and so on.
 - Why check the signature first: without it, anyone who guesses the URL could send patients fake "appointment cancelled" emails.
 - Why reply 202 before sending emails: the backend gets its answer in milliseconds; slow email sending happens after.

- Why one Switch: adding a new event later is one more output, not a new workflow.
- Better structure once it works: move each branch into its own workflow and call it with **Execute Workflow**. Small workflows are easier to test and reuse.

Workflow 3: Reminder emails the day before

1. **Schedule Trigger:** every day at 18:00.
2. **HTTP Request:** `GET {{$env.DOCOP_API_URL}}/api/internal/appointments/due-reminders, query date = {{$now.plus({days: 1}).toFormat('d_M_yyyy')}}, header x-internal-key = {{$env.DOCOP_INTERNAL_KEY}}.`
3. **Split Out:** field `appointments`, so each appointment becomes its own item.
4. **HTTP Request:** `POST .../appointments/{{$json.id}}/reminder-sent` (same header).
5. **IF:** `{{$json.firstTime}}` is true.
6. **Brevo:** "Reminder: your appointment with Dr. ... tomorrow at ...".
 - Why `d_M_yyyy`: it matches your `slotDate` format exactly (`1_10_2026`, no leading zeros).
 - Why mark as sent **before** emailing: if the workflow runs twice (a manual test plus the schedule), the second run gets `firstTime: false` and nobody gets two reminders.

Workflow 4: Weekly admin report

1. **Schedule Trigger:** Mondays at 08:00.
2. **HTTP Request:** `GET .../api/internal/stats/weekly` with the key header.
3. **Brevo** to the admin: booked, cancelled, completed and paid counts, plus the cancellation rate `{{Math.round($json.cancelled / Math.max($json.booked, 1) * 100)}}%`.
 - Why: this shows n8n doing reporting, not just notifications, which is a different business use.

Workflow 5 (optional, strong for your CV): AI triage of chat messages

Once the others work, use what you learned in the n8n AI Agents course: on a new patient message event, an **AI Agent** node labels urgency (routine / soon / urgent) and emails the doctor only for urgent ones.

- Keep it to labelling. Never let the AI give medical advice to patients.

Step 4: Make failures visible and safe

Production systems fail. The difference is whether you find out, and whether a retry makes things worse.

1. One global Error Workflow

- Create a workflow: **Error Trigger** → **Brevo** email to you with `{{ $json.workflow.name }}`, `{{ $json.execution.error.message }}` and `{{ $json.execution.url }}`.
- In **every** other workflow: **Settings** → **Error workflow** → pick it.
- Why: without it, a failed run just sits in the executions list and nobody looks. With it, every failure reaches your inbox with a link to the exact run.

2. Retries on outside calls

- On every HTTP Request and Brevo node: **Settings** → **Retry on fail**, max tries 3, wait 5000 ms.
- Why: most failures are brief (a network blip, Brevo rate limit). A retry fixes them without waking you up.

3. Idempotency (safe to run twice)

- Reminders: already handled by `reminderSentAt` (step 2c).
- Event emails: the event `id` is unique. If you ever see duplicates, store sent ids and skip repeats.
- Why: retries plus manual test runs mean "exactly once" can't be guaranteed. "At most once per appointment" is the practical goal, and you built it into the data.

4. The backend never depends on n8n

- Test it: `docker compose stop n8n`, then book an appointment. It must still succeed; the backend log shows `[event] appointment.booked not delivered`.
- Why: this is the whole point of fire-and-forget. Say this in interviews.

5. Known gap, and the next level

- If n8n is down, that event is lost (the reminders workflow still catches reminders later).
- The professional fix is an **outbox**: save events in a Mongo `events` collection first, and a small job retries unsent ones. Mention it as "next improvement"; building it is optional.

6. Protect the timezone logic

- Your server may run in UTC, but patients live in Sri Lanka time. Keep all schedule times in n8n (`GENERIC_TIMEZONE`), never hard-code offsets in workflows.

Step 5: Workflows in Git, tests, and your CV

5a. Keep workflows in the repo

Workflows built only in the n8n screen exist only in your Postgres volume. Export them into Git so anyone can recreate them.

1. Add a folder mount to the `n8n` service in `docker-compose.yml` :

```
volumes:
  - n8n-data:/home/node/.n8n
  - ./n8n/workflows:/home/node/workflows
```

2. Export after every change:

```
docker compose exec n8n n8n export:workflow --all --separate --
output=/home/node/workflows
```

3. On a fresh machine, import them:

```
docker compose exec n8n n8n import:workflow --separate --
input=/home/node/workflows
```

- Why: version history, code review on workflow changes, and one-command setup, the same as your Dockerfiles.
- Safe to commit: secrets are `$env....` references and credentials are only names. Open one JSON file and check before your first commit anyway.

5b. Add backend tests (you already have Jest)

- `test/internal.test.js` : calling `/api/internal/stats/weekly` with no key returns **401**; a wrong key returns **401**; the right key returns **200**.
- `test/events.test.js` : mock `fetch` and check that `emitEvent` sends `x-docop-signature` equal to your own HMAC of `timestamp.body`.
- Why: security code without tests is a claim; with tests it is evidence. Reviewers look for this.

5c. Done checklist

- ☐ n8n image pinned; pruning and timezone set; two new secrets in `.env` and `.env.example`
- ☐ `eventService.js`, `authInternal.js`, `internalRoute.js` added; `reminderSentAt` in the model
- ☐ `emitEvent` called after book, pay, cancel (3 places) and complete
- ☐ Internal route returns 401 without the key
- ☐ Workflow 1 health monitor emails you when you stop the backend
- ☐ Workflow 2 returns 401 to a request with a wrong signature, and sends booking emails for a real booking
- ☐ Workflow 3 sends one reminder per appointment, even when run twice
- ☐ Workflow 4 weekly report arrives
- ☐ Error workflow attached to every workflow; retries on every HTTP and Brevo node
- ☐ Booking still works with n8n stopped
- ☐ Old `sendEmail` calls in `verifyMockPayment` removed after workflow 2 works
- ☐ Workflows exported to `n8n/workflows/`; tests pass; README has an "Automation" section with the diagram idea

5d. CV lines

Designed an event-driven automation layer for a MERN telemedicine app with n8n: HMAC-signed webhooks with replay protection, a key-protected internal API, and fire-and-forget delivery so core booking never depends on automation.

Automated appointment confirmations, idempotent day-before reminders, uptime monitoring and weekly reporting; centralised error alerting and retries; workflows version-controlled in Git and deployed with Docker Compose.

After this comes Phase 3: the MCP server, which lets an AI assistant use these same internal APIs safely.